

# **Algorithmique avancée et programmation**

---

CERI L2 Informatique

**BAROUDI TOUFIK**

# Programme

---

- Introduction
- Chapitre1: Les arbres
- Chapitre 2: Le Tas (Heap)
- Chapitre 3: Les algorithmes de tri
- Chapitre 4: La complexité algorithmique
- Chapitre 5: Backtracking,
- Chapitre 6: Programmation dynamique
- Chapitre 7: Les tables de hachage



# Introduction

## Définition:

Un **algorithme** se définit comme une suite finie d'opérations élémentaires pour la résolution d'un problème donné.

## Définition:

L'**algorithmique** est l'ensemble des méthodes et des techniques permettant de **concevoir** et d'**analyser** un algorithme.

# Introduction

## Définition:

L'**analyse** d'un algorithme consiste :

- ❖ À prouver que la méthode proposée résout bien le problème donné en entrée (**preuve de l'algorithme**),
- ❖ À évaluer le temps d'exécution et l'espace mémoire requis (**complexité en temps et en espace**).

## Définition:

Un **programme** informatique est le codage d'un algorithme dans un langage informatique donné (C, C++, Java, Python,...).

# Introduction



## Ecriture des algorithmes:

- Un algorithme doit pouvoir être programmé dans **n'importe quel langage informatique**.
- Nous décrirons les algorithmes par des “pseudo-codes”.
- Les implémentations se feront en **C++ en TP**.

# Introduction

## Plan du cours:

|   | Chapitre                 | CM | TD | TP |
|---|--------------------------|----|----|----|
| 1 | Arbres                   | 2  | 2  | 2  |
| 2 | Tas                      | 1  | 1  | 1  |
| 3 | Tris                     | 1  | 1  | 1  |
|   | CC1                      |    |    |    |
| 4 | Complexité algorithmique | 1  | 1  | 1  |
| 5 | Backtracking             | 1  | 1  | 1  |
| 6 | Programmation dynamique  | 1  | 2  | 2  |
| 7 | Tables de hachage        | 1  | 1  | 1  |
|   | CC2                      |    |    |    |

## TP Codes C++:

localement ou sur Caseine

## Note Algorithmique avancée:

$0,4 * \text{CC1}(\text{écrit}) + 0,4 * \text{CC2}(\text{écrit}) + 0,2 * \text{TPs}$



# **Chapitre 1:** **Les arbres**

---

# Chapitre1: Les arbres

## Introduction

### Définition:

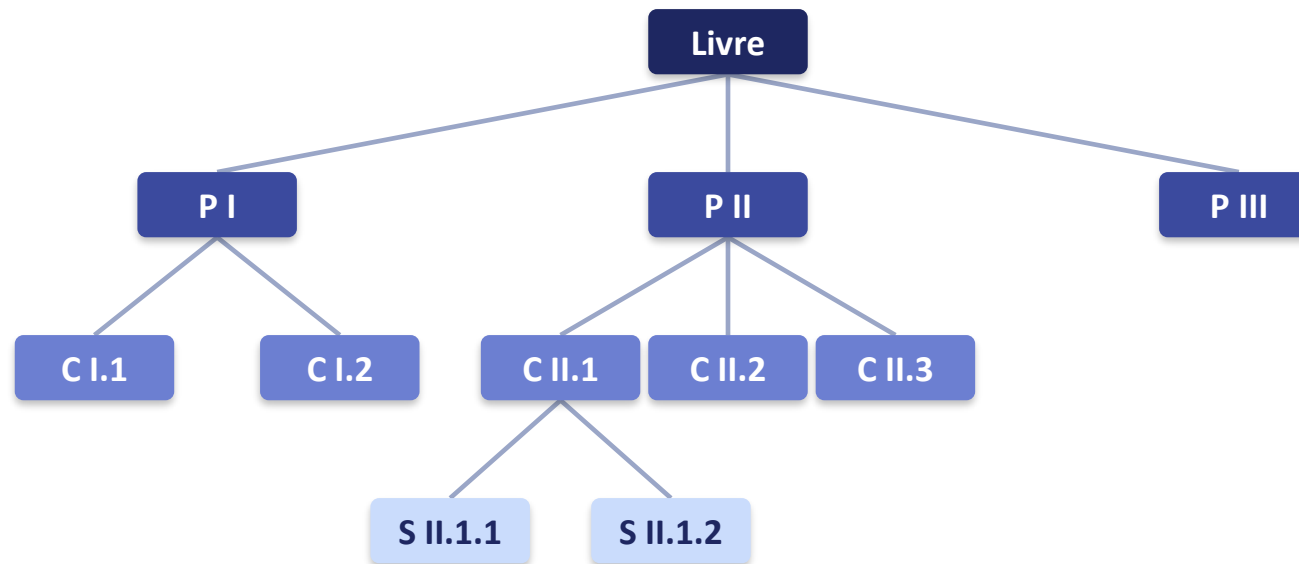
Un **arbre** est un ensemble de nœuds organisés de façon hiérarchique à partir d'un nœud appelé racine.

## Introduction

Leur usage est multiple, car il capte l'idée de hiérarchie.

Exemple:

- ❖ Découpage d'un livre en parties, chapitres, sections, paragraphes...,

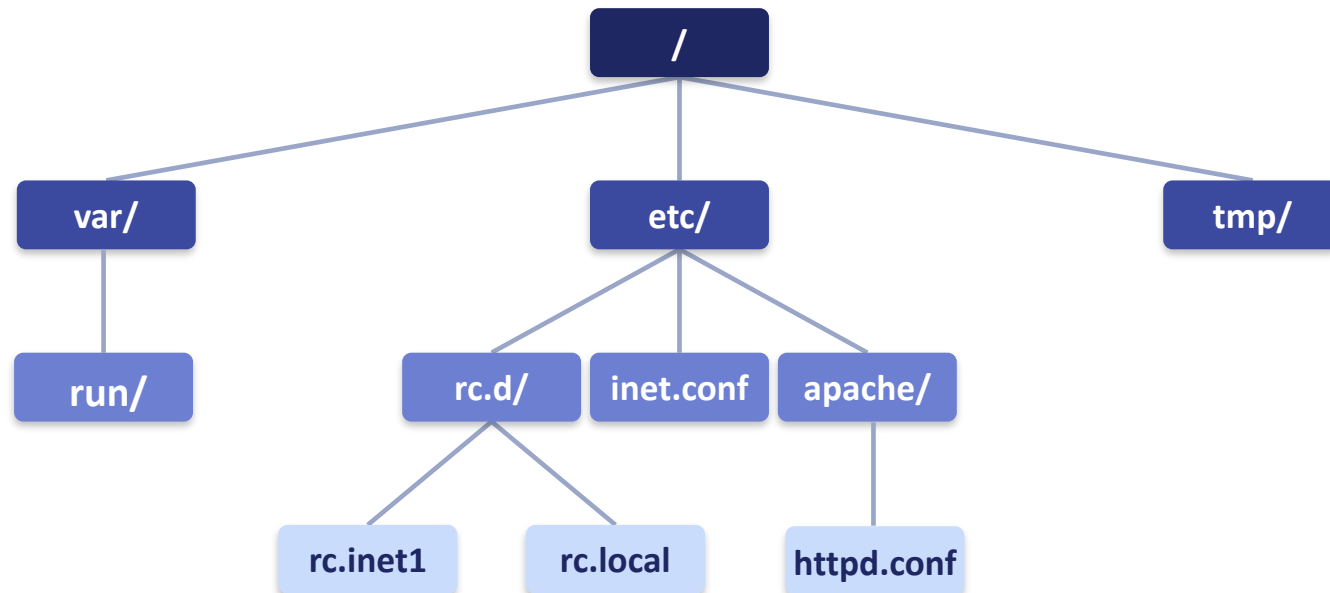


## Introduction

Leur usage est multiple, car il capte l'idée de hiérarchie.

Exemple:

❖ Hiérarchies de fichiers.



# Chapitre1: Les arbres

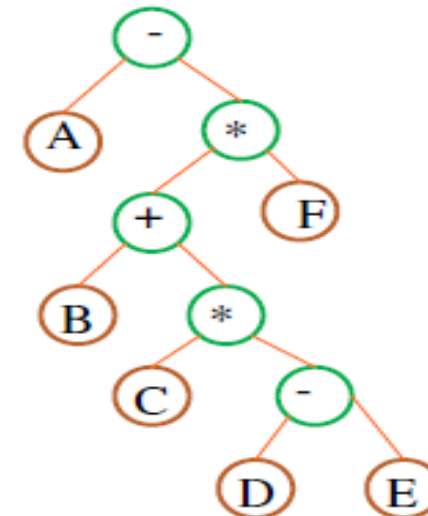
## Introduction

Leur usage est multiple, car il capte l'idée de hiérarchie.

### Exemple:

#### ❖ Expressions arithmétiques.

- L'expression  $A - (B + C * (D - E)) * F$  se représente facilement par un arbre où apparaît clairement la priorité des opérations:

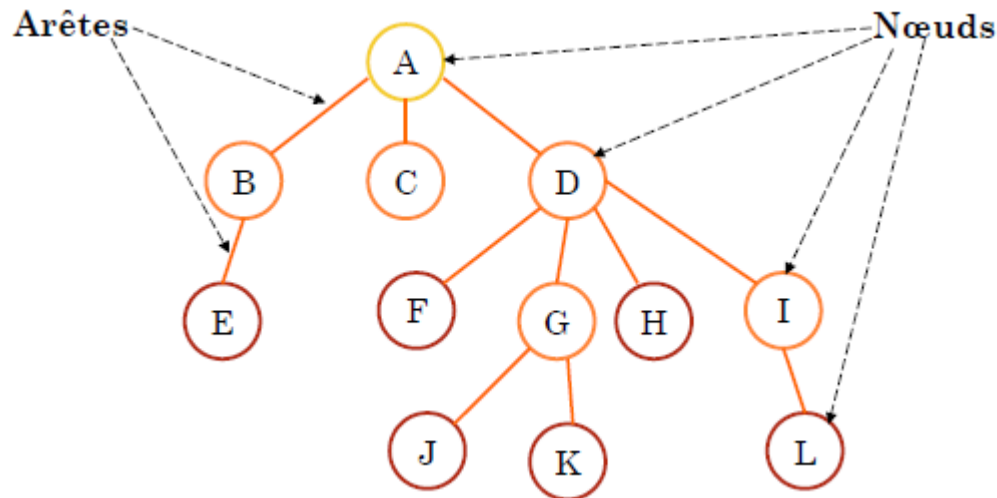


# Chapitre1: Les arbres

## Définitions & Terminologies



Un arbre est une structure de données (souvent dynamique) représentant un ensemble de valeurs organisées hiérarchiquement (non linéaire). Chaque valeur est stockée dans un **nœud**. Les nœuds sont connectés entre eux par des **arêtes** qui représentent des relations parent/fils.



# Chapitre1: Les arbres

## Définitions & Terminologies



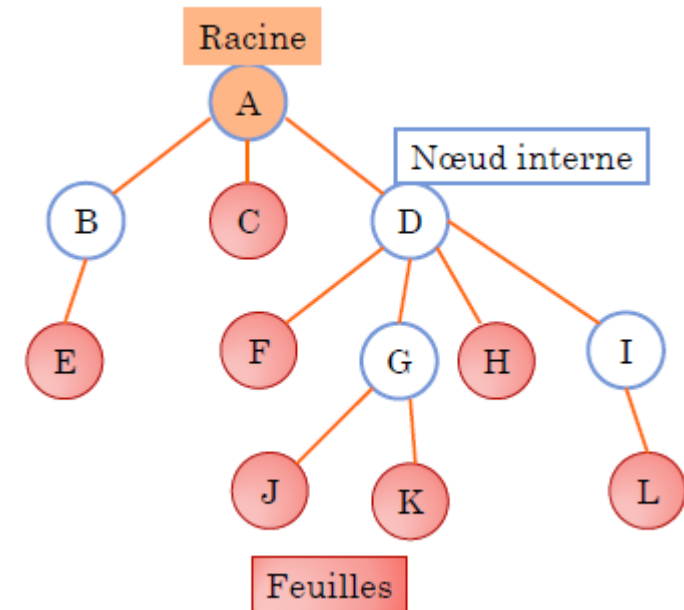
**Racine:** est le nœud qui n'a pas de **prédécesseur** (parent) et possède zéro ou plusieurs **fils**. La racine constitue la caractéristique d'un arbre.



**Feuille :** est un nœud qui n'a pas de **successeur** (fils). Une feuille est aussi appelée un nœud externe.



**Noeud interne :** est tout nœud qui admet au moins un successeur (fils).



# Chapitre1: Les arbres

## Définitions & Terminologies



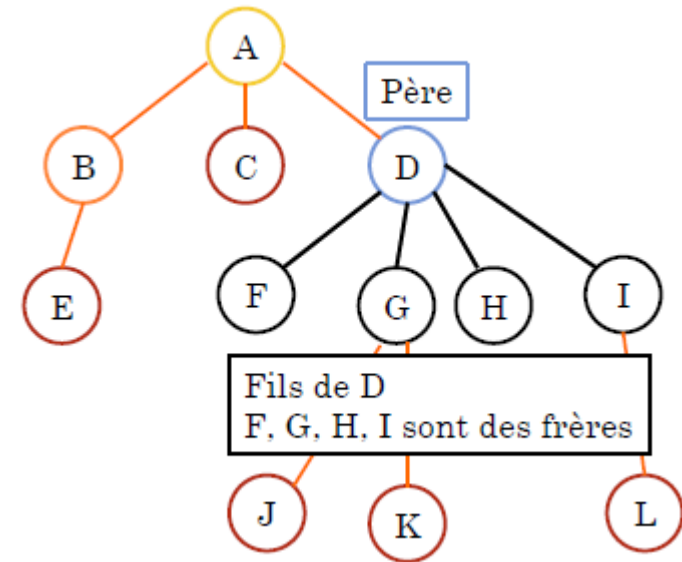
**Père** : est un nœud qui admet au moins un successeur (fils). Dans l'exemple, D est le père des nœuds F, G, H et I.



**Fils d'un nœud** : sont ses successeurs. Dans l'exemple, F, G, H, et I sont les fils du nœud D.



**Frères** : sont les successeurs ou les fils issus d'un même nœud (parent direct). Dans l'exemple, F, G, H et I sont des frères.



## Définitions & Terminologies

### Définition:

Un arbre, enraciné en  $r$ , à **ramification bornée** peut être défini récursivement comme suit:

$$A = \emptyset \text{ ou } A = \langle r, F_1, F_2, \dots, F_k \rangle$$

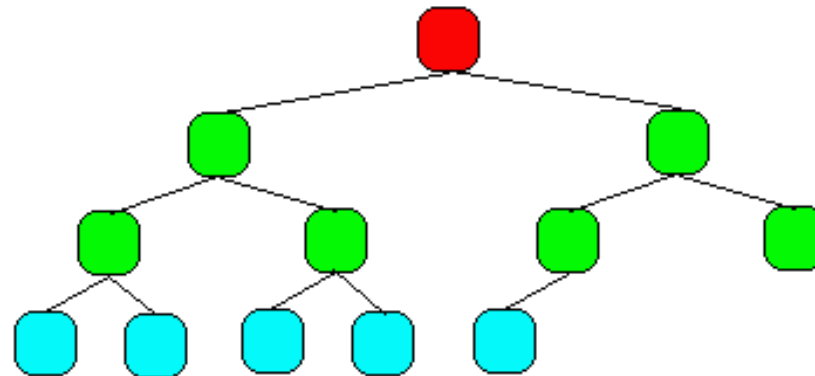
où ' $r$ ' est un nœud appelé racine et  $F_1, F_2, \dots, F_k$  sont des arbres disjoints appelés **sous-arbres** de  $A$ .

Les nœuds contiennent plusieurs champs dont **une étiquette** (identifiant) nommée "**cle**" (**r.cle**).

## Définitions & Terminologies

### Arbre binaire, $K=2$ :

- F1 : sous-arbre gauche, sa racine est le fils gauche de r.
- F2 : sous-arbre droit, sa racine est le fils droit de r.
- Noeud simple : noeud interne n'ayant qu'un fils.
- Noeud double : noeud interne ayant deux fils.



## Représentation des AB

| Noeud |                                 |
|-------|---------------------------------|
| cle   | un identifiant                  |
| pere  | adresse du nœud père            |
| fg    | adresse du nœud dit fils gauche |
| fd    | adresse du nœud dit fils droit  |

Pour un nœud  $x$ ,  $x.cle$ ,  $x.pere$ ,  $x.fg$ ,  $x.fd$  désignent les champs.

# Chapitre1: Les arbres

## Codage des AB en c++

```
class noeud
{
    friend class AB;
    int cle ;
    noeud * fg ;
    noeud * fd ;
    noeud * pere ;
public :
    noeud(int x) ;
    ~noeud() ;
    ...
} ;
```

```
class AB
{
    noeud * r ;
public :
    AB(noeud * x) ;
    ~AB() ;
    ...
} ;
```

“**friend class AB**” : toutes les méthodes de la classe AB peuvent accéder aux attributs privés de “noeud”.

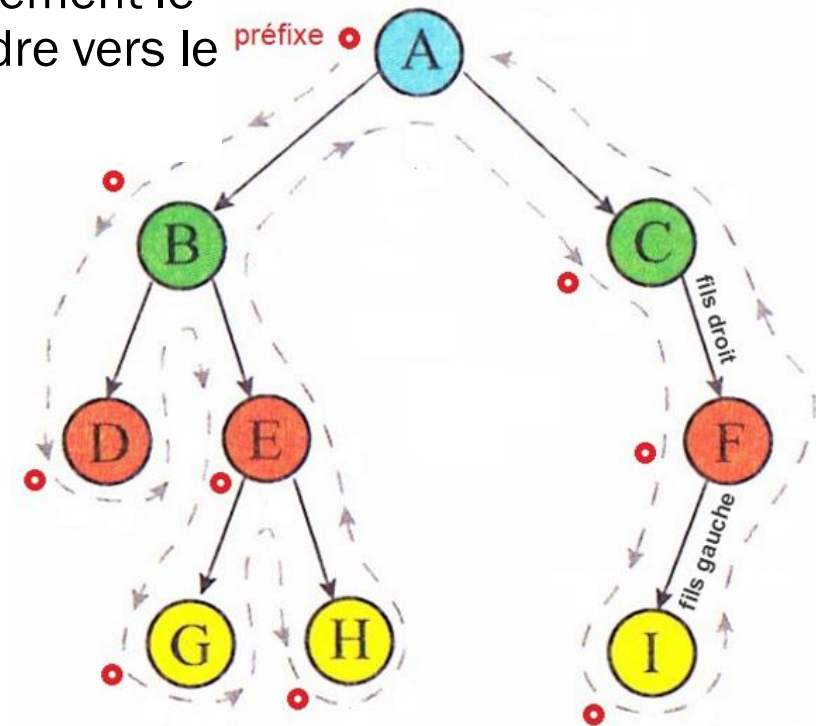
# Chapitre1: Les arbres

## Parcours dans un AB

### Parcours Préfixe:

On part de la racine puis on visite son fils gauche (et éventuellement le fils gauche de celui-ci, etc.) avant de remonter et de redescendre vers le fils droit.

```
Algorithme: prefixe(r)  
Si (r ≠ null) alors  
    affichage r.cle;  
    prefixe(r.fg);  
    prefixe(r.fd);  
Fin
```



A B D E G H C F I

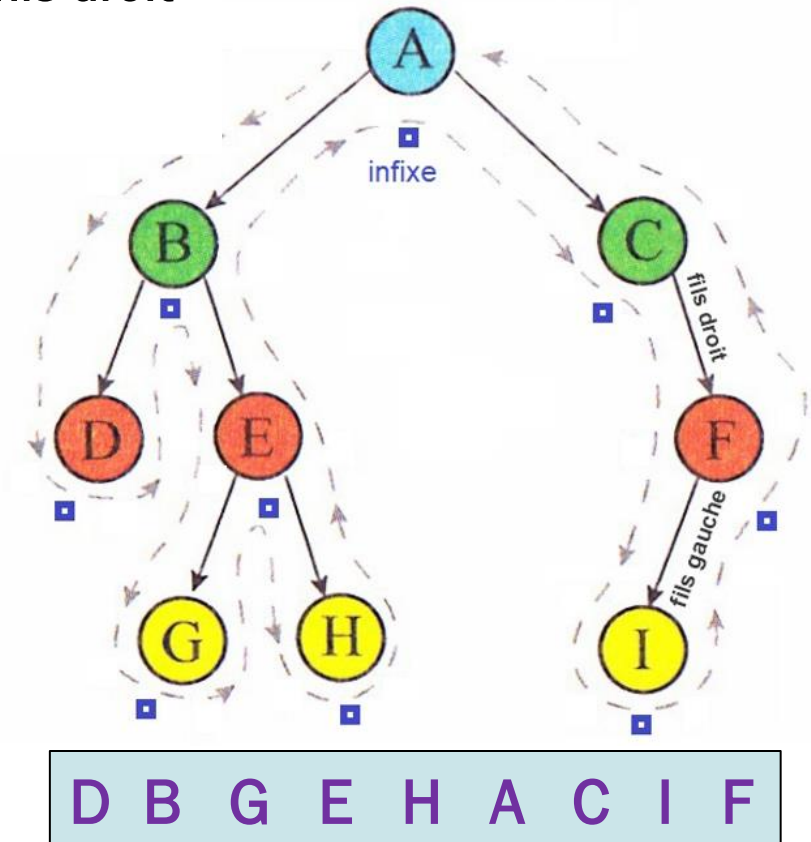
# Chapitre1: Les arbres

## Parcours dans un AB

### Parcours Infixe:

Chaque nœud est visité après son fils gauche mais avant son fils droit.

```
Algorithme: infixe(r)
Si (r≠ null) alors
    infixe(r.fg);
    affichage r.cle;
    infixe(r.fd);
Fin
```



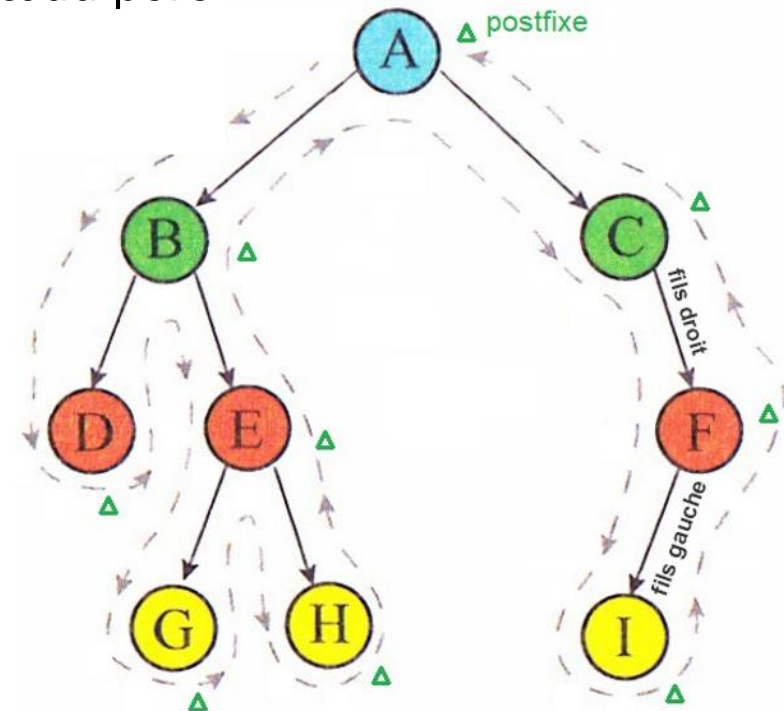
# Chapitre1: Les arbres

## Parcours dans un AB

### Parcours Postfixe:

On part de la **feuille la plus à gauche**, et on ne remonte à un nœud père que si ses fils ont tous été visités.

```
Algorithme: postfixe(r)
Si (r≠ null) alors
    postfixe(r.fg);
    postfixe(r.fd);
    affichage r.cle;
Fin
```



D G H E B I F C A

# Chapitre1: Les arbres

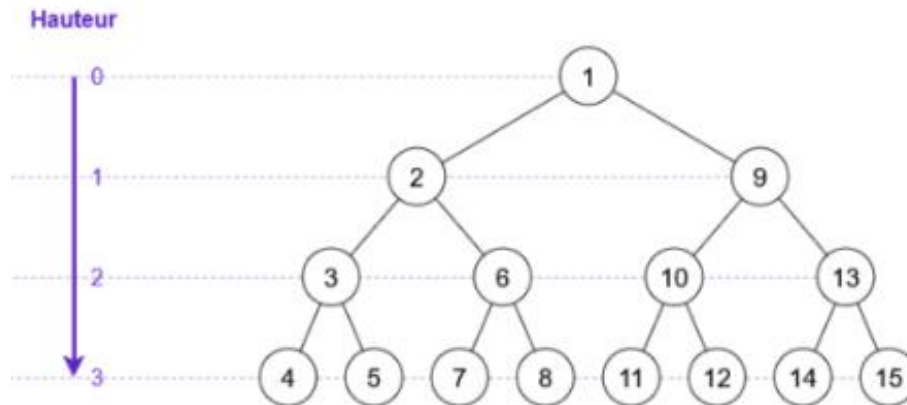
## Hauteur

**Définition:** On appelle **hauteur**  $h$ , ou **profondeur**, d'un nœud  $x$  d'un arbre binaire  $B$  de racine  $r$ , la fonction définie récursivement comme suit :

- $h(r) = 0$
- $h(x) = 1 + h(y)$ , si  $y$  est le père de  $x$

**Définition:** On appelle **hauteur**  $h$ , ou **profondeur**, d'un arbre binaire  $B$ , la valeur:

- $h(B) = \max \{ h(x) \mid x \text{ est un nœud de } B \}$



Hauteur de l'arbre :  $h(B) = 3$

# Chapitre1: Les arbres

## Hauteur

### Définition:

Un arbre binaire B dont tous les nœuds (autre que les feuilles) sont doubles est dit **complet**.

### Théorème:

Un arbre binaire B est complet ssi il a  $2^i$  nœuds de hauteur i pour i allant de 0 à  $h(B)$ . Le nombre total de ses nœuds est  $2^{h(B)+1} - 1$

### Théorème:

Soit B un arbre binaire de n nœuds et de hauteur  $h(B)$ . On a:

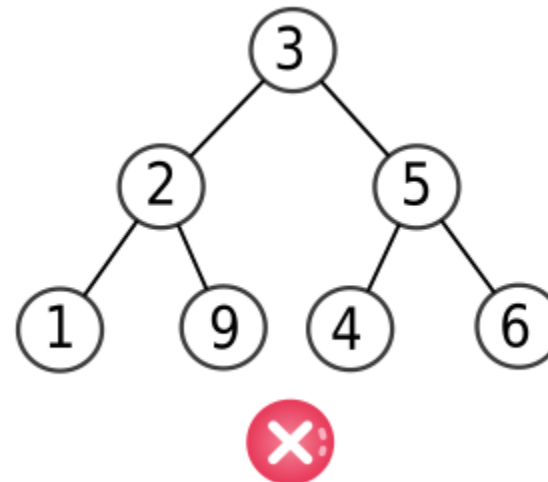
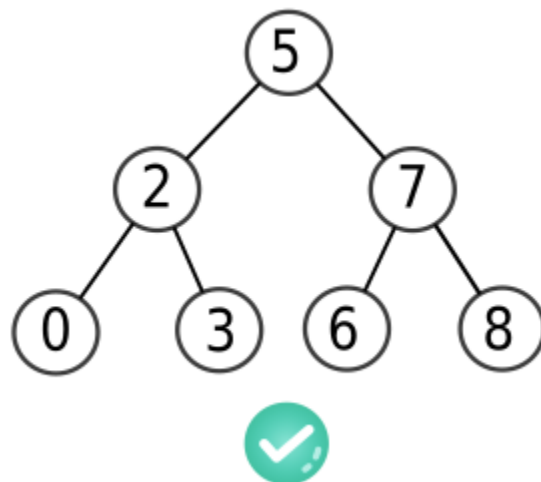
$$\log_2(n) \leq h(B) \leq n - 1$$

## Arbre binaire de recherche

### Définition:

Un **Arbre Binaire de Recherche (ABR)** est un arbre binaire ordonné tel que pour tout nœud  $\ll i \gg$ :

- ❑ Toutes les valeurs du sous arbre gauche de  $\ll i \gg$  sont strictement inférieures à la valeur de  $\ll i \gg$ , et
- ❑ Toutes les valeurs du sous-arbre droit de  $\ll i \gg$  sont supérieures ou égales à la valeur de  $\ll i \gg$ .



## Recherche dans un ABR

La recherche d'un nœud ayant une clé particulière « x » se fait de manière récursive:

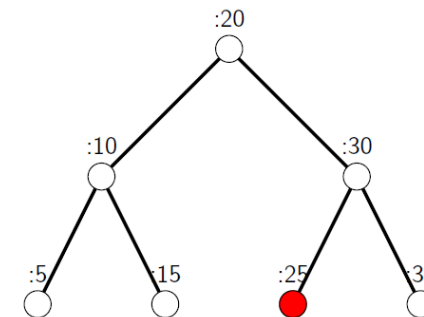
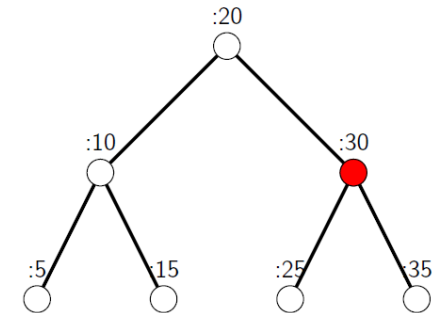
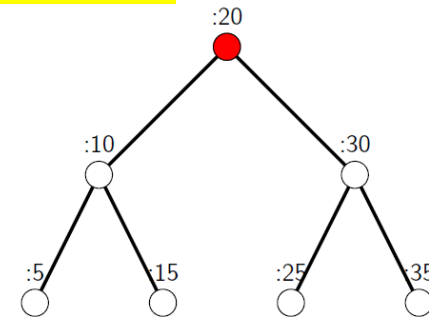
- ☐ Si l'arbre est vide: **échec**
  - ✓ **Recherche négative:** la clé n'appartient pas à l'arbre, **l'algorithme se termine.**
- ☐ Sinon: comparer la clé « x » avec la clé de la racine.
- ☐ S'il y a égalité alors la clé « x » est **trouvée**.
  - ✓ **Recherche positive:** **l'algorithme se termine.**
- ☐ Si la clé « x » est strictement inférieure, alors la recherche est poursuivie dans le sous arbre gauche.
- ☐ Si la clé « x » est strictement supérieure, alors la recherche est poursuivie dans le sous arbre droit.

# Recherche dans un ABR

## Chapitre1: Les arbres

Chercher 25

```
Algorithme: recherche(r,k)
Si (r== null) ou (k== r.cle) alors
    retourner(r);
Fin
Si (k < r.cle) alors
    retourner(recherche(r.fg,k));
Fin
Sinon
    retourner(recherche(r.fd,k));
Fin
```

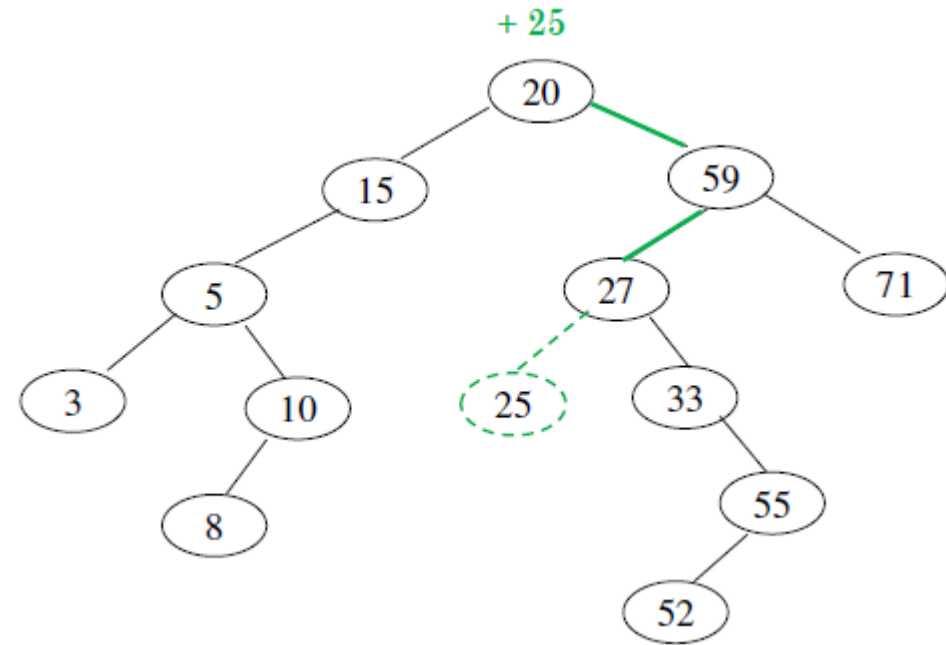


# Chapitre1: Les arbres

## Opération d'insertion

L'insertion d'une clé « x » se fait toujours au niveau d'une **feuille**. Cette insertion dans un **ABR** doit maintenir la propriété d'ordre des arbres de recherche, ainsi:

- ☐ Rechercher la position d'insertion
- ☐ Raccorder le nouveau nœud a son parent



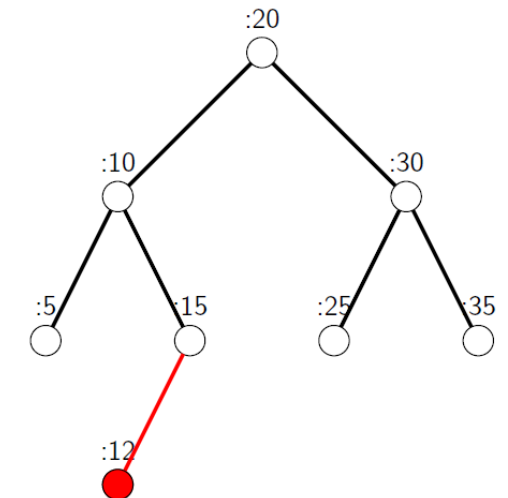
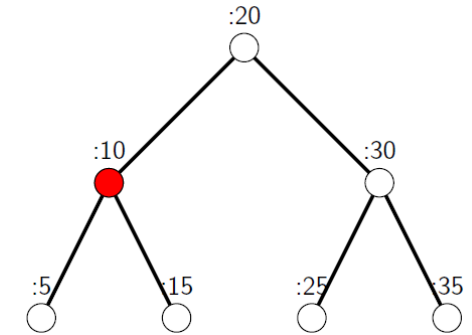
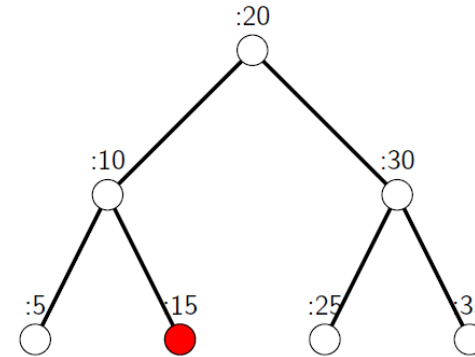
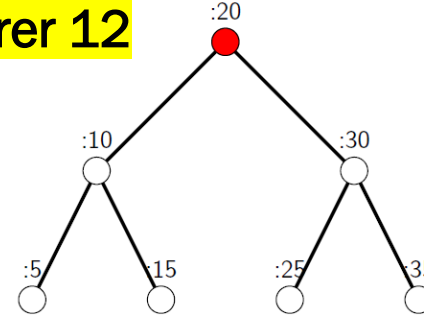
# Opération d'insertion

## Chapitre1: Les arbres

Algorithme: `insertionFeuille(r,y)`

```
x = r , pere = null;  
tant que (x ≠ null) faire  
  Pere = x;  
  Si (y.cle ≤ x.cle) alors  
    x = x.fg;  
  Fin  
  Sinon  
    x = x.fd;  
  Fin  
Fin  
y.pere = pere;  
Si (y.pere == null) alors  
  r = y;  
Fin  
Sinon  
  Si (y.cle ≤ y.pere.cle) alors  
    y.pere.fg = y;  
  Fin  
  Sinon  
    y.pere.fd = y;  
  Fin  
Fin
```

Insérer 12



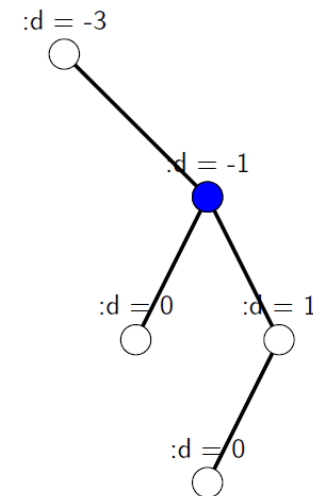
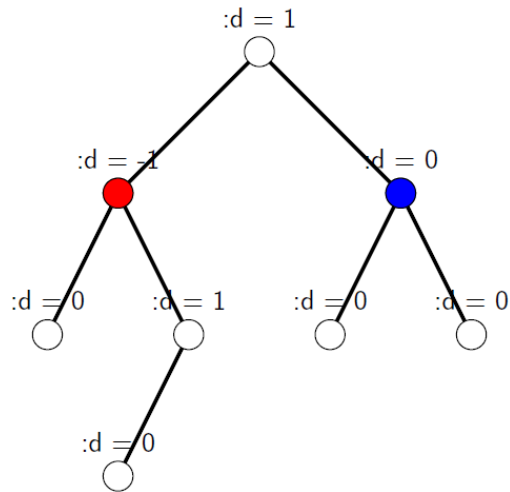
# Arbres équilibrés de type AVL

Adelson – Velskii - Landis

## Chapitre1: Les arbres

### Définition:

Soit  $T$  un arbre binaire de racine  $r$ , on appelle **déséquilibre** de  $T$  la différence  $d(r) = h(G) - h(D)$  où  $h(G)$  (resp.  $h(D)$ ) désigne la hauteur du sous-arbre de racine  $r.fg$  (resp.  $r.fd$ ).



# Arbres équilibrés de type AVL

Adelson – Velskii - Landis

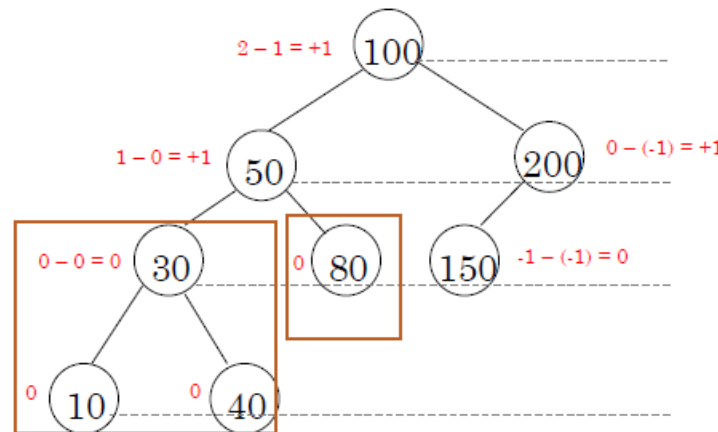
## Chapitre1: Les arbres

### Définition:

T est dit **équilibré** (resp. **déséquilibré**) ssi  $|d(v)| \leq 1$  (resp  $|d(v)| \geq 2$ ) en tout nœud v de T.

### Définition:

Un **AVL** est un arbre binaire de recherche équilibré.



Cet arbre est un arbre AVL

# Chapitre1: Les arbres

## AVL-Techniques d'équilibrage

L'opération d'équilibrage, appelée **rotation**, s'applique à tous les **ABR** dans le but de pouvoir les rééquilibrer. Il y en a 4 fondamentales :

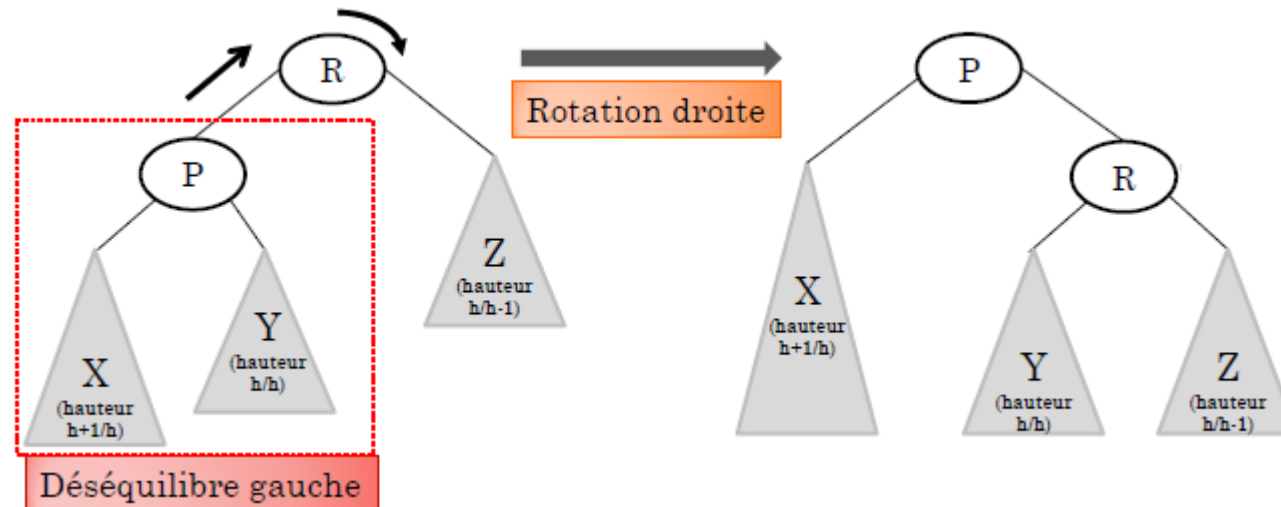
- ☐ rotation droite
- ☐ rotation gauche
- ☐ rotation gauche-droite
- ☐ rotation droite-gauche

## AVL-Techniques d'équilibrage

### 1. Rotation droite:

Soit  $A=(B,R,Z)$  un ABR tel que  $B=(X, P, Y)$ .

✓ La rotation droite est l'opération:  $((X, P, Y), R, Z) \rightarrow (X, P, (Y, R, Z))$



# AVL-Techniques d'équilibrage

## Chapitre1: Les arbres

$d(x) = 2$  et  $d(x.fg) = 1$

Rotation droite sur un nœud  $x$  se trouvant dans un arbre enraciné en  $r$ .  $x$  supposé non nul ici.

Algorithme: **rotationDroite(x)**

$p = x.pere$  ,  $G = x.fg$ ;

$G.pere = p$  ,  $x.fg = G.fg$ ;

**Si** (  $G.fg \neq \text{null}$  ) **alors**

$G.fg.pere = x$ ;

**Fin**

$G.fg = x$ ;

$x.pere = G$ ;

**Si** (  $p == \text{null}$  ) **alors**

$r = G$ ;

**Fin**

**Sinon**

**Si** (  $p.fg == x$  ) **alors**

$p.fg = G$ ;

**Fin**

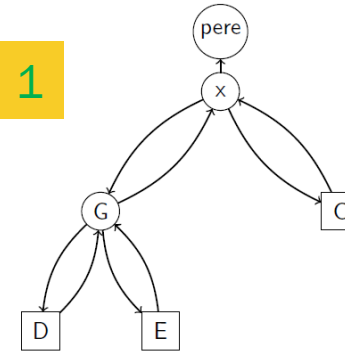
**Sinon**

$p.fg = G$ ;

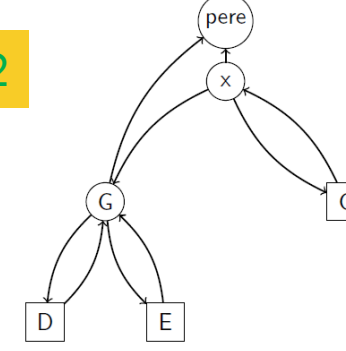
**Fin**

**Fin**

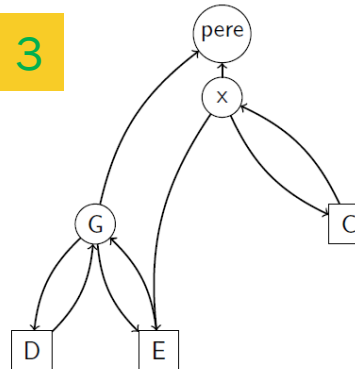
1



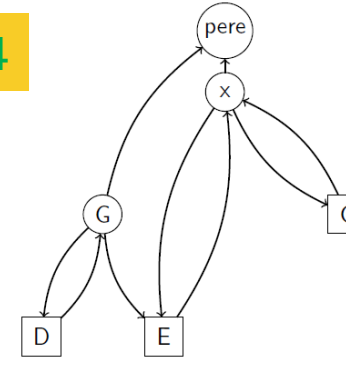
2



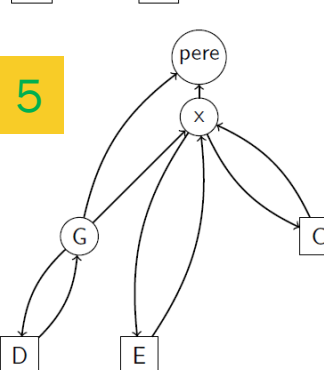
3



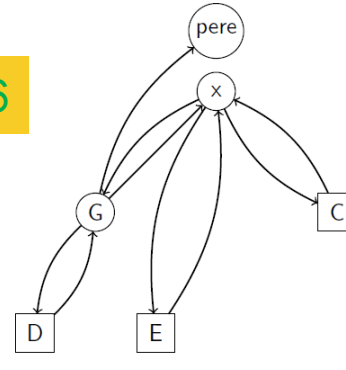
4



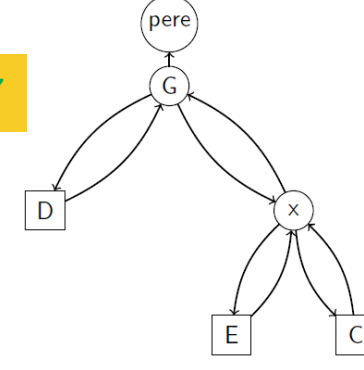
5



6



7



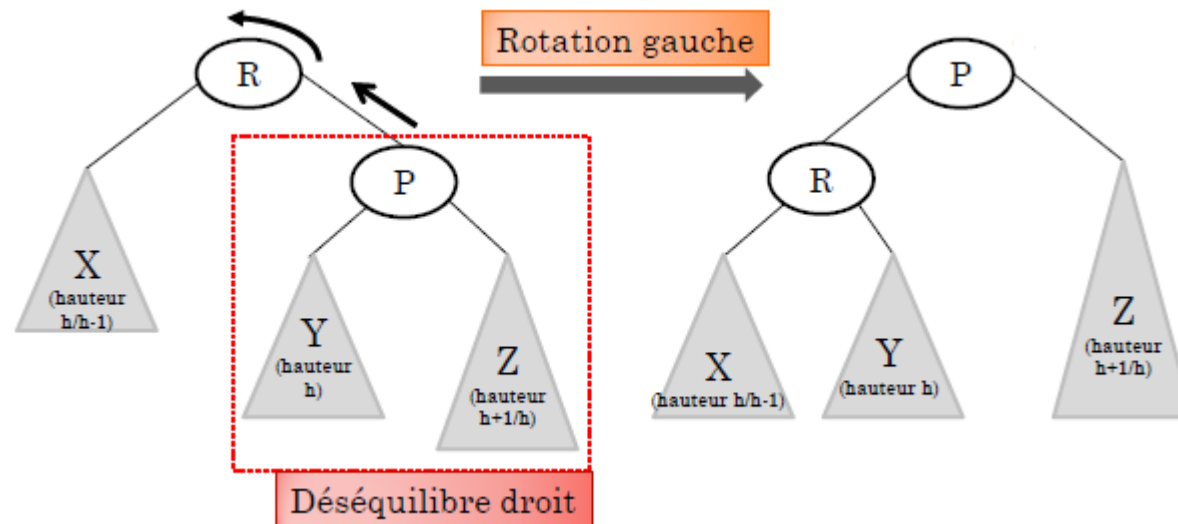
# Chapitre1: Les arbres

## AVL-Techniques d'équilibrage

### 2. Rotation gauche:

Soit  $A=(X,R,B)$  un ABR tel que  $B=(Y, P, Z)$ .

✓ La rotation gauche est l'opération:  $(X, R, (Y, P, Z)) \rightarrow ((X, R, Y), P, Z)$



$d(x) = -2$  et  $d(x.fg) = -1$

Algorithme: `rotationGauche(x)`

Symétrique de la rotation droite : Remplacer G par D et fg par fd partout.

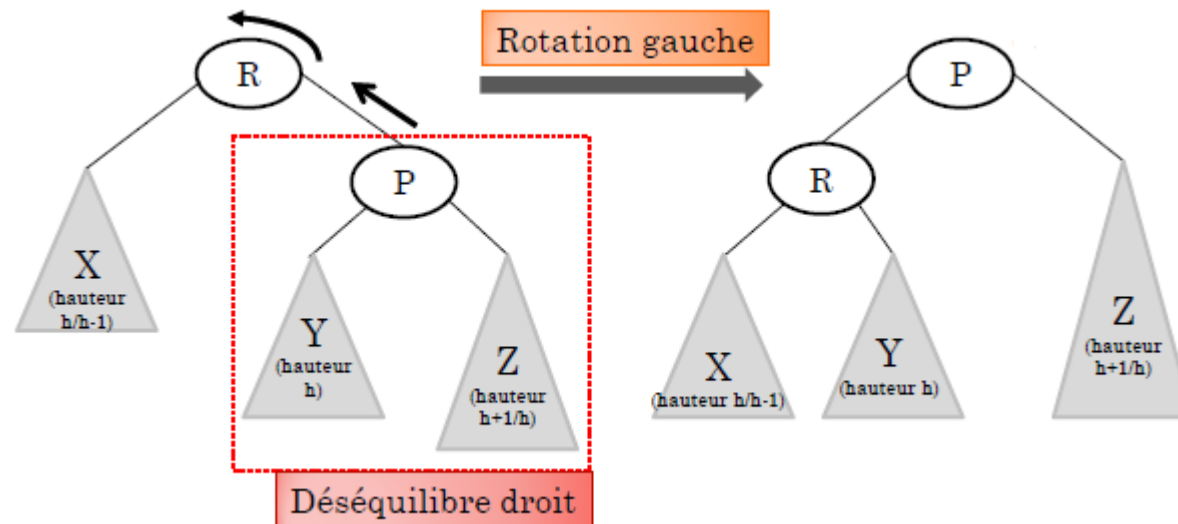
# Chapitre1: Les arbres

## AVL-Techniques d'équilibrage

### 2. Rotation gauche:

Soit  $A=(X,R,B)$  un ABR tel que  $B=(Y, P, Z)$ .

✓ La rotation gauche est l'opération:  $(X, R, (Y, P, Z)) \rightarrow ((X, R, Y), P, Z)$



$d(x) = -2$  et  $d(x.fg) = -1$

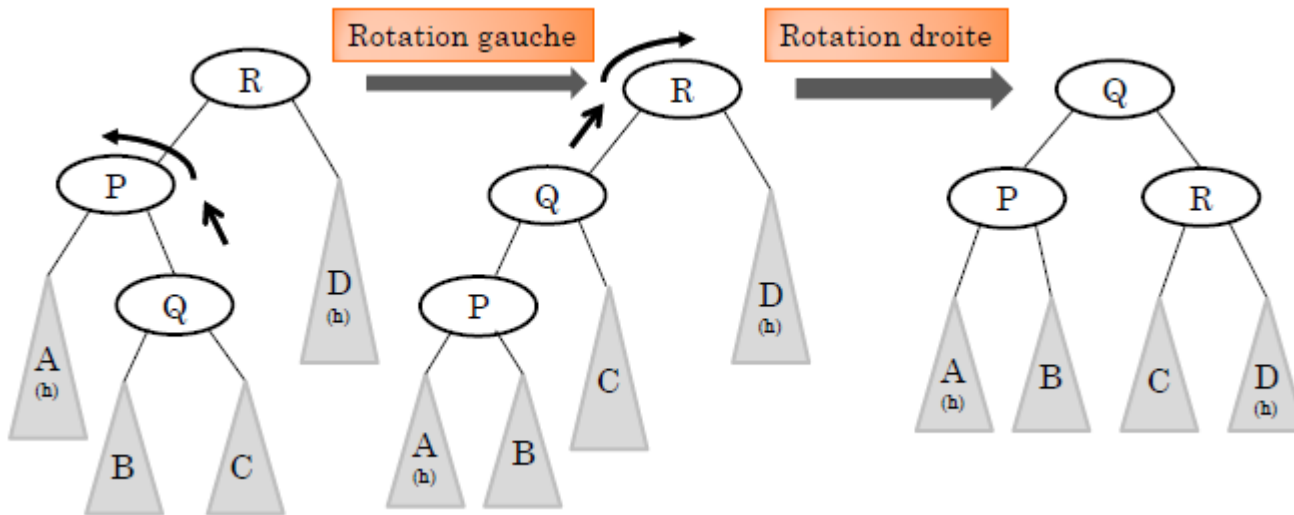
Algorithme: `rotationGauche(r,x)`

Symétrique de la rotation droite : Remplacer G par D et fg par fd partout.

## AVL-Techniques d'équilibrage

### 3. Rotation gauche-droite:

C'est une rotation gauche sur le sous arbre-gauche du nœud P suivie d'une rotation droite sur le nœud R



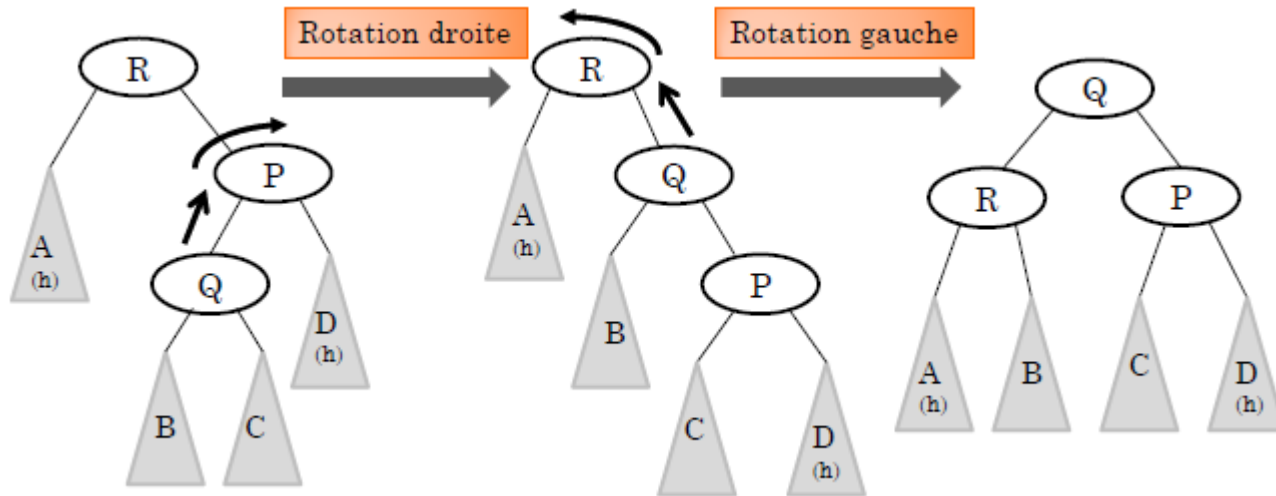
$$d(x) = 2 \text{ et } d(x.fg) = -1$$

Algorithme: `rotationGaucheDroite(x)`  
`rotationGauche(x.fg);`  
`rotationDroite(x);`

## AVL-Techniques d'équilibrage

### 4. Rotation droite-gauche:

C'est une rotation droite sur le sous arbre-droit du nœud P suivie d'une rotation gauche sur le nœud R



$$d(x) = -2 \text{ et } d(x.fg) = 1$$

Algorithme: `rotationDroiteGauche(x)`  
`rotationDroite(x.fd);`  
`rotationGauche(x);`

## **AVL-Insertion**

- L'insertion d'un élément dans un arbre AVL peut provoquer un déséquilibre.
- L'ajout d'un nœud se fait toujours au niveau d'une feuille, puis on rééquilibre l'arbre AVL si l'insertion a déséquilibré l'arbre.
- Le déséquilibre est rencontré lorsque le facteur d'équilibrage d'un nœud de l'arbre égale  $\pm 2$ .

# Chapitre1: Les arbres

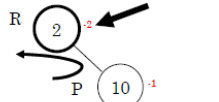
## AVL-Insertion

**Exemple:** Soit la série de nombres à insérer dans un arbre AVL  
(2 10 12 4 16 8 6 14 9 1 7)

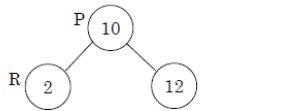
2 10 12 4 16 8 6 14



2 10 12 4 16 8 6 14

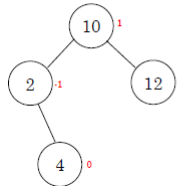


*Rotation simple à gauche*



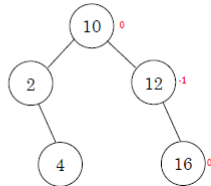
1

2 10 12 4 16 8 6 14



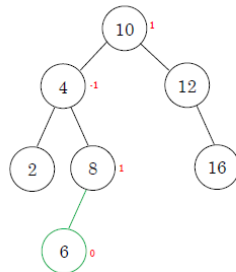
2

2 10 12 4 16 8 6 14



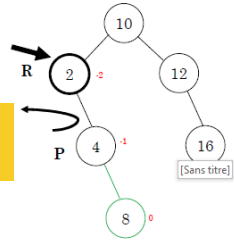
4

2 10 12 4 16 8 6 14 9 1 7

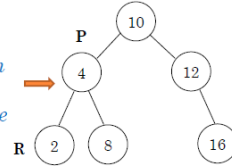


4

2 10 12 4 16 8 6 14 9 1 7

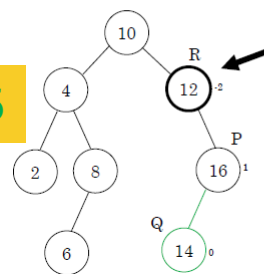


*Rotation simple à gauche*

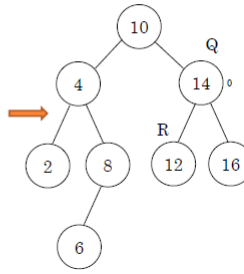


3

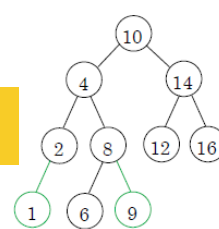
2 10 12 4 16 8 6 14 9 1 7



*Rotation double Droite gauche*

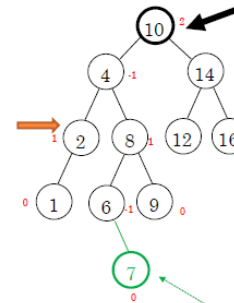


2 10 12 4 16 8 6 14 9 1 7



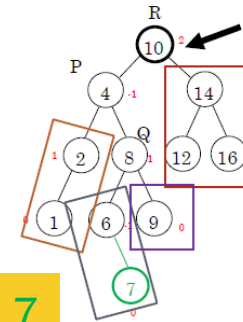
6

*Insertion du 7*

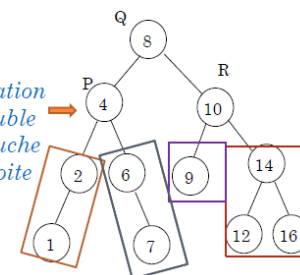


7

2 10 12 4 16 8 6 14 9 1 7



*Rotation double Gauche droite*



# Chapitre1: Les arbres

## AVL - Insertion

```
Algorithme: insertionFeuilleEquilibre(r,y)
insertionFeuille(r,y);
Si (y.pere ≠ null) alors
    requilibre(y.pere);
Fin
```

- ❖ Un rééquilibrage peut être fait par la méthode “requilibre(v)”.
- ❖ Tout nœud “v” possède deux autres attributs : “h” (hauteur de l’arbre enraciné en v), “d” déséquilibre de l’arbre enraciné en “v”.
- ❖ Les méthodes de rotation doivent mettre à jour “h” et “d”.

```
Algorithme: requilibre(v)
anch = v.h ;
Si (v.fg == null) alors
    hg = -1; dg = 0 ;
Fin
Sinon
    hg = v.fg.h; dg = v.fg.d ;
Fin
Si (v.fd == null) alors
    hd = -1; dd = 0 ;
Fin
Sinon
    hd = v.fd.h; dd = v.fd.d ;
Fin
v.d = hg - hd ; v.h = max(hg , hd) +1;
Si (v.d == 2) et (dg == 1) alors
    rotationDroite(v);
Fin
Sinon Si (v.d == -2) et (dd == -1) alors
    rotationGauche(v);
Fin
Sinon Si (v.d == 2) et (dg == -1) alors
    rotationGaucheDroite(v);
Fin
Sinon si (v.d == -2) et (dd == 1) alors
    rotationDroiteGauche(v);
Fin
Si (anch ≠ v.h) et (v.pere ≠ null) alors
    requilibre(v.pere);
Fin
```

## Hauteur d'un arbre équilibré

### Théorème:

Tout arbre équilibré de  $n$  nœuds a une hauteur  $h$  vérifiant :

$$\log_2(n + 1) \leq h + 1 \leq 1.44 \cdot \log_2(n + 2)$$

## Hauteur d'un arbre équilibré

### Théorème:

Tout arbre équilibré de  $n$  nœuds a une hauteur  $h$  vérifiant :

$$\log_2(n + 1) \leq h + 1 \leq 1.44 \cdot \log_2(n + 2)$$

### Preuve:

1. L'AVL de hauteur  $h$  comportant le plus nœuds est un arbre complet. Son nombre de nœuds est  $2^{h+1} - 1$ . On a donc:

$$n \leq 2^{h+1} - 1,$$

$$n + 1 \leq 2^{h+1},$$

$$\log_2(n + 1) \leq h + 1.$$

## Hauteur d'un arbre équilibré

### Preuve (suite):

2.

- L'AVL de hauteur  $h$  comportant le moins nœuds est un arbre dans lequel le déséquilibre de chaque nœud interne est égal à 1 ou  $-1$ . Soit  $N(h)$  le nombre de nœuds minimal pour un tel AVL.
- On a:  $N(0) = 1$ ,  $N(1) = 2$ .
- Pour obtenir un arbre AVL de hauteur  $h$  avec le moins de nœuds possible, l'un des sous-arbres doit avoir la hauteur maximale autorisée  $h-1$ , et l'autre sous-arbre doit avoir une hauteur de  $h-2$ . On a donc la relation de récurrence suivante :

$$N(h) = 1 + N(h - 1) + N(h - 2), h \geq 2$$

- On en déduit que :

$$N(h) + 1 = 2 + N(h - 1) + N(h - 2) = (N(h - 1) + 1) + (N(h - 2) + 1)$$

- En posant  $F(h) = N(h) + 1$ , on obtient la suite de Fibonacci suivante :

$$F(h) = F(h - 1) + F(h - 2), h \geq 2.$$

avec  $F(0) = 2$  et  $F(1) = 3$ .

## Hauteur d'un arbre équilibré

### Preuve (suite):

2.

- Nous savons (voir outils mathématiques) que  $F(h)$  peut s'écrire :

$$F(h) = \frac{1}{\sqrt{5}} (r_1^{h+3} - r_2^{h+3})$$

$$\text{Où } r_1 = \frac{1+\sqrt{5}}{2} \quad \text{et } r_2 = \frac{1-\sqrt{5}}{2}$$

- Donc :  $(n+1) \geq N(h) + 1 = F(h) = \frac{1}{\sqrt{5}} (r_1^{h+3} - r_2^{h+3})$

$$(n+1) \geq \frac{1}{\sqrt{5}} (r_1^{h+3} - 1) \quad \text{car } |r_2^{h+3}| < 1$$

$$\sqrt{5}(n+1) + 1 \geq r_1^{h+3}$$

$$\log_{r_1}(\sqrt{5}(n+1) + 1) \geq h+3$$

- Comme  $\sqrt{5} > 1$  et  $\log$  est une fonction croissante on a:

$$\log_{r_1}(\sqrt{5}(n+2)) > \log_{r_1}(\sqrt{5}(n+1) + 1) \geq h+3$$

- Or :  $\log_{r_1}(\sqrt{5}(n+2)) = \log_{r_1}(\sqrt{5}) + \log_{r_1}(n+2)$

$$\text{Donc: } \log_{r_1}(\sqrt{5}(n+2)) = \log_{r_1}(\sqrt{5}) + \frac{\log_2(n+2)}{\log_2(r_1)}$$

## Hauteur d'un arbre équilibré

### Preuve (suite):

2.

- D'autre part,  $\log_{r_1}(\sqrt{5}) < 2$ . On en déduit donc que:

$$2 + \frac{\log_2(n+2)}{\log_2(r_1)} \geq h + 3$$

$$\frac{\log_2(n+2)}{\log_2(r_1)} \geq h + 1$$

- Tenant compte du fait que:  $\frac{1}{\log_2(r_1)} \cong 1.44$ . On a finalement:

$$1.44 \cdot \log_2(n + 2) \geq h + 1$$